

AI Coding Tutor

Kody Takada

Electrical and Computer Engineering

University of Michigan

Ann Arbor, United States of America

ktakada@umich.edu

Abstract—Understanding programming concepts remains a significant challenge for many students, especially when existing tools prioritize solution delivery over conceptual clarity. This project introduces an interactive AI coding tutor designed to bridge that gap by combining code summarization and natural language explanation generation. The system employs a two-stage architecture: CodeT5-small generates concise summaries of user-provided code, and TinyLlama-1.1B produces beginner-friendly explanations in response to natural language questions, using both the code and its summary as contextual input. A fully deployed web application provides a user-facing interface for real-time interaction. Evaluation against ChatGPT-4.0 using standard metrics demonstrates that while the system’s lexical similarity to reference explanations is low, its high BERTScore indicates strong semantic alignment. These results underscore the effectiveness of lightweight, modular models in delivering educationally valuable explanations. The project lays the groundwork for future development in adaptive, explainable AI tools for programming education.

I. INTRODUCTION

A. Overview

This project aims to develop an interactive web-based AI coding tutor that assists users in understanding programming concepts through natural language explanations and visual aids. The application will allow users to input code snippets and receive step-by-step, line-by-line explanations in plain English, with a particular emphasis on foundational logic, use of control structures, and abstract programming concepts such as recursion or functional composition.

The system architecture consists of two main components: a frontend web application and a backend powered by advanced language models. The frontend will serve as the user interface, offering an intuitive platform for code input, explanation viewing, and execution flow visualization.

The backend will leverage a combination of pretrained encoder-decoder large language models (LLMs) for code understanding and natural language generation. Specifically, the system will integrate CodeT5, a model trained on code–text pairs and well-suited for tasks such as summarization and explanation, and FLAN-T5, a general-purpose instruction-tuned model from Google, capable of adapting to a wide range of user prompts. By fine-tuning and orchestrating these models, the system aims to generate pedagogically sound explanations and context-aware responses tailored to learners of varying experience levels.

Ultimately, the goal is to bridge the gap between static code generation tools and human tutoring by providing a guided, explainable, and learner-centered coding experience.

B. Background and Motivation

As a computer science tutor at the University of Michigan, I frequently assist students who struggle with understanding both basic and advanced programming concepts. Many face difficulties in interpreting code snippets, debugging errors, or internalizing functional programming paradigms. While existing tools such as GitHub Copilot, ChatGPT, and Stack Overflow offer rapid code suggestions and answers, they often fall short in providing beginner-friendly explanations. These platforms tend to prioritize efficiency over pedagogy, delivering solutions without fostering deep comprehension.

Through my tutoring experience, I have observed students repeatedly encounter the same issues—even after being presented with a working solution—highlighting a gap between access to answers and genuine understanding. This underscored the need for a tool that merges the precision of AI with the clarity and empathy of human tutoring.

This project seeks to address that gap by developing an interactive AI coding tutor capable of explaining code line-by-line in plain, accessible language. The tool emphasizes foundational principles, addresses “why” questions (e.g., “Why is recursion appropriate here?”), and incorporates visualizations to demystify abstract concepts such as control flow and recursion. By shifting the focus from simply solving problems to promoting conceptual mastery, the project aims to empower students to become more confident and independent programmers.

C. Prior Work

The landscape of AI-assisted programming education has advanced along two primary dimensions: (1) code understanding models that generate explanations and (2) pedagogical frameworks aimed at effective knowledge transfer.

On the technical front, CodeT5 introduced a transformer-based architecture pretrained on code–text pairs, using docstrings to facilitate code summarization and documentation generation [1]. Their identifier-aware attention mechanism enabled nuanced code feature extraction—a technique this project intends to adopt by fine-tuning on curated explanation datasets.

For conversational tutoring, PLBART demonstrated that denoising pretraining objectives, such as code masking and fragment reconstruction, enhanced the model’s ability to understand and reason about program logic [2]. This directly informs our approach to handling student queries that require explanation of code functionality.

From an educational perspective, studies such as Prather et al. have evaluated the role of AI tools like GitHub Copilot in classrooms [3]. While such tools improved productivity, they were found to be insufficient in fostering conceptual understanding of core programming principles. These findings can be extended to other state-of-the-art large language models (LLMs), including ChatGPT, DeepSeek, and others.

Despite ongoing progress in the field, critical gaps remain in areas such as dynamic visualization (e.g., animating recursive function calls), learner adaptation (e.g., customizing responses based on student background), and conversational repair (e.g., addressing misunderstandings in follow-up queries).

II. METHOD

A. Problem Formulation

The primary objective of this project is to construct a system capable of generating pedagogically useful explanations for programming code in response to natural language questions. The learning problem can be framed as a conditional text generation task: given a source code snippet and a natural language query, generate a detailed and beginner-friendly explanation that promotes conceptual understanding.

Input: A user-provided code snippet C and a natural language question Q related to the code.

Output: A natural language explanation A that is contextually accurate, clear, and pedagogically aligned with the user’s intent.

Dataset: For training and inference, the models leverage open-source datasets and pretrained knowledge. While no custom dataset was fine-tuned in this iteration, the models were chosen based on their pretraining corpora, which include large-scale code–text pairs (for CodeT5) and diverse instruction-following datasets (for TinyLlama). Optionally, small, manually annotated explanation samples were used for testing and evaluation.

Model Formulation: The model architecture follows a multi-stage pipeline:

- 1) Code Understanding Stage – A transformer-based encoder-decoder model (CodeT5-small) is used to generate a concise but informative summarization S of the input code C .
- 2) Response Generation Stage – A lightweight instruction-following model (TinyLlama-1.1B) takes as input the original code C , the user’s question Q , and the generated summary S , and outputs the explanation A .

Mathematically, the problem is to learn a function f such that:

$$A = f(C, Q, S), \quad \text{where } S = g(C) \quad (1)$$

and g is the code summarization function modeled by CodeT5.

B. Methodology

To address the task of interactive coding instruction, this project implements a two-stage pipeline that combines the complementary strengths of CodeT5 and TinyLlama:

1) Stage 1 - Code Summarization with CodeT5

The system first employs CodeT5-small, a pretrained transformer model optimized for source code summarization, to generate a high-level description of the input code. This summary provides structural and semantic context that improves the model’s understanding of the code beyond just syntax.

2) Stage 2- Explanation Generation with TinyLlama

The explanation generation step uses **TinyLlama-1.1B**, a compact, instruction-tuned LLM. This model is prompted with the original code snippet, the user’s question, and the CodeT5-generated summary. The system constructs a single prompt that contextualizes the code and question, enhancing the model’s ability to reason and respond accurately. The combined context allows TinyLlama to tailor its explanation to both the functional purpose of the code and the pedagogical intent of the question.

3) Design Considerations

- This modular design separates code analysis from response generation, enabling flexibility in model selection and fine-tuning.
- The summarization step reduces noise and enhances focus for the generative model.
- Prompt engineering plays a key role in ensuring consistent output quality from TinyLlama.

Future iterations may include fine-tuning on educational QA datasets, incorporating feedback loops, and adapting the prompt dynamically based on user background (e.g. novice vs intermediate).

III. RESULTS

A. Model Setup and Data Pipeline

The model pipeline evaluated consists of a two-stage architecture: CodeT5-small is used for summarizing the input code, followed by TinyLlama-1.1B for generating an explanation in response to a natural language question. The pipeline takes the original code, a student-style question, and the generated summary as input to produce a context-aware answer. For evaluation, outputs from this pipeline were compared to those generated by ChatGPT-4.0 using identical prompts.

B. Evaluation Metrics and Figure

The performance of the AI coding tutor was assessed using four standard text generation metrics: ROUGE-1, ROUGE-2, ROUGE-L, BLEU, and BERTScore (F1). These metrics capture both lexical and semantic similarity with respect to human-written explanations.

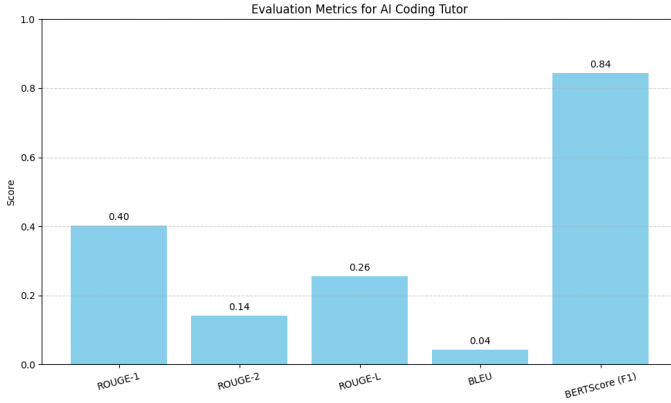


Fig. 1. Comparison of output quality from the CodeT5-TinyLlama pipeline against ChatGPT-4.0 across several evaluation metrics. Each shows the average score of the AI Coding tutor relative to reference explanations.

C. Interpretation of Results

The results from figure 1 highlight trade-off between surface-level phrasing and semantic correctness. While the lexical overlap scores - ROUGE-1 (0.40), ROUGE-2 (0.14), ROUGE-L (0.26), and BLEU (0.04) - are relatively low, the BERTScore (0.84) is substantially higher. This suggests that although the AI coding tutor does not replicate the exact wording of reference explanations, it captures the underlying meaning and intent effectively.

In contrast to ChatGPT-4.0, which tends to produce more fluent and stylistically refined responses, the CodeT5-TinyLlama pipeline prioritizes semantic fidelity through structured prompts and code summarization. The high BERTScore indicates that the model successfully communicates the same concepts as human explanations, however, using different wording.

These findings reinforce the potential of using smaller, more efficient models in instructional contexts, especially when guided by carefully constructed prompts and modular design. Despite its compact size, the pipeline produces explanations that are semantically aligned with ground-truth responses, making it a viable tool for educational use where interpretability and accuracy of ideas are more important than perfect fluency.

IV. FUTURE WORK

While the current system demonstrates promising results in generating semantically accurate explanations, several directions exist for enhancing its effectiveness and scope:

- 1) **Fine-Tuning on Educational Data:** The current models rely solely on pretrained knowledge. Fine-tuning on curated datasets of beginner-level explanations and code-question pairs could improve alignment with educational objectives and clarity.
- 2) **Execution Flow Visualization:** Future iterations could incorporate code execution visualizations (e.g., variable tracing, call stack animation) to help users better understand abstract concepts like recursion and scope.

- 3) **Multilingual Support:** Extending the system to support explanations in other languages would broaden its accessibility for non-English-speaking learners.

These enhancements would deepen the system's educational value and support broader deployment in academic and self-learning environments.

V. CONCLUSION

This project presents an end-to-end AI coding tutor that combines the strengths of pretrained code-aware and instruction-tuned language models to generate interactive, pedagogically grounded code explanations. By integrating CodeT5-small for code summarization and TinyLlama-1.1B for natural language explanation generation, the system delivers personalized and semantically rich responses to user queries. Evaluation results demonstrate that while the generated responses may diverge from reference explanations in syntax, they remain strongly aligned in meaning—an important characteristic for educational applications.

A key outcome of this work is the successful deployment of the AI coding tutor as a fully functional web application, allowing real-time interaction with users in an accessible and scalable format. The application not only facilitates code comprehension through plain-language explanations but also opens avenues for future enhancements such as adaptive tutoring, execution visualizations, and expanded language support.

In summary, this project bridges the gap between static AI coding tools and human-centered learning by emphasizing understanding over rote output. It sets the foundation for building lightweight, explainable, and learner-focused AI systems in programming education.

REFERENCES

- [1] Wang et al. (2021). CodeT5: Identifier-aware Unified Pretraining for Code. EMNLP
- [2] Ahmad et al. (2021). PLBART: Unified Pretraining for Program Understanding. ACL
- [3] Prather et al. (2023). Github Copilot in Classrooms: Productivity vs. Understanding. SIGCSE
- [4] Jain et al. (2023). EXPLAN: Human-Evaluated Code Explanation. NeurIPS